Exploratory Data Analysis: Electronics Market

End-to-End Data Cleaning, Feature Engineering, and Statistical Analysis using MySQL

Project Overview

This project demonstrates a comprehensive approach to data wrangling and statistical analysis directly within a relational database. Working with a raw, unstructured electronics retail dataset, the objective was to transform messy text fields into actionable quantitative metrics, followed by univariate and bivariate statistical modeling to uncover market trends.

Data Transformation Scope

Original Feature	Challenge	Engineered Solution
Price	Mixed formats (e.g., "$209.99 through Special Event")	Extracted clean numeric values via REGEXP.
Discount	Text conditions (e.g., "After $400 OFF")	Calculated exact MRP; categorized discount types.
Rating	Merged sentences (e.g., "Rated 4.8 out of 5... 125 reviews")	Split into two discrete columns: Avg_Rating & Review_Count.
Title	Lengthy product names and varied formatting	Extracted standardized Brand Names for group analysis.

Executive Summary

Through advanced SQL string manipulation, regex pattern matching, and custom stored procedures, an unstructured dataset was successfully normalized. Statistical analysis revealed a right-skewed pricing distribution and quantified a positive correlation (slope of 19.26) between product ratings and review volume.

Phase 1: Advanced Data Cleaning

Raw data often contains anomalies, currency symbols, and appended promotional text that prevents mathematical operations. I utilized MySQL's Regular Expression capabilities to surgically extract quantitative data from unstructured text strings.

1. Cleaning the Price Column

To extract pure decimal values from strings containing diverse promotional text, I implemented `REGEXP_SUBSTR` paired with a pattern that captures optional decimal points.

```
-- Extracting pure numeric values from messy Price strings UPDATE ELECTRONICS_DATA SET
PRICE = CAST(REGEXP_SUBSTR(Price, '[0-9]+(\\.[0-9]+)?') AS DECIMAL(10,2)) WHERE PRICE
LIKE '%THROUGH SPECIAL EVENT%';
```

2. Feature Engineering: Ratings & Reviews

The original `RATING` column contained full sentences. I engineered two new numerical columns, extracting the core rating and isolating the review count using positive lookahead logic.

```
-- 1. Extract the base rating (e.g., '4.8') UPDATE ELECTRONICS_DATA SET AVG_RATING =
REGEXP_SUBSTR(RATING, '[0-9]+(\\.[0-9]+)?'); -- 2. Extract the review count explicitly
looking for numbers before 'reviews' UPDATE ELECTRONICS_DATA SET REVIEWS_COUNT =
REGEXP_SUBSTR(RATING, '[0-9]+(?= reviews)');
```

> **Data Integrity Achieved**
>
> By dropping obsolete categorical columns (like Currency) and standardizing brands (grouping Mac/iPad under "Apple"), the dataset was primed for complex statistical aggregations without data-type mismatch errors.

Phase 2: Univariate Analysis & Outlier Detection

Because MySQL lacks a native `PERCENTILE()` function, calculating the Interquartile Range (IQR) for outlier detection requires a custom approach. I developed a Stored Procedure utilizing window functions to dynamically fetch any requested percentile.

1. Custom Stored Procedure for Percentiles

```
DELIMITER $$ CREATE PROCEDURE GETRPICEBYPERCENTILE( IN PERCENTILEVALUE DECIMAL(3,2), OUT PRICE_LIMIT DECIMAL(10,2) ) BEGIN SELECT MAX(PRICE) INTO PRICE_LIMIT FROM( SELECT PRICE, ROUND(PERCENT_RANK() OVER (ORDER BY PRICE), 2) AS PERCENTILES FROM ELECTRONICS_DATA ) K WHERE PERCENTILES = PERCENTILEVALUE; END $$ DELIMITER ; -- Fetching Quartiles CALL GETRPICEBYPERCENTILE(0.25, @Q1); -- Result: 726.28 CALL GETRPICEBYPERCENTILE(0.75, @Q3); -- Result: 2209.00
```

2. IQR Outlier Filtering

Using the variables generated by the stored procedure, I applied the standard statistical IQR formula to isolate items priced abnormally high or low.

```
SELECT PRICE FROM ELECTRONICS_DATA WHERE PRICE < (@Q1 - 1.5 * (@Q3 - @Q1)) OR PRICE > (@Q1 + 1.5 * (@Q3 - @Q1));
```

> **Distribution Insight**
>
> The IQR filter isolated prices like $2,998.81 and $2,983.93. Because outliers existed exclusively on the higher end of the spectrum, it indicates the electronics pricing distribution is **positively skewed** (right-skewed).

Phase 3: Bivariate Statistical Modeling

To understand the relationship between a product's rating and its volume of reviews, I calculated the statistical Slope and Covariance from scratch using foundational mathematical formulas translated into SQL.

1. Calculating the Trend Line (Slope)

Slope represents the rate of change. I calculated this by finding the sum of the product of deviations from the mean for both variables, divided by the sum of squared deviations of the independent variable.

```
-- Formula: sum((x - avg(x)) * (y - avg(y))) / sum((x - avg(x))^2) SELECT SUM(
(AVG_RATING - (SELECT AVG(AVG_RATING) FROM ELECTRONICS_DATA)) * (REVIEWS_COUNT -
(SELECT AVG(REVIEWS_COUNT) FROM ELECTRONICS_DATA)) ) / SUM( POWER(AVG_RATING - (SELECT
AVG(AVG_RATING) FROM ELECTRONICS_DATA), 2) ) AS SLOPE FROM ELECTRONICS_DATA;
```

> **Interpreting the Slope (Output: 19.26)**
>
> A slope of 19.26 indicates that for every **1-point increase in a product's average rating**, we can generally expect to see an increase of about **19 additional reviews**. This highlights a clear correlation: higher-rated products tend to accumulate more customer engagement.

2. Covariance Validation

```
SELECT SUM( (AVG_RATING - (SELECT AVG(AVG_RATING) FROM ELECTRONICS_DATA)) *
(REVIEWS_COUNT - (SELECT AVG(REVIEWS_COUNT) FROM ELECTRONICS_DATA)) ) / (COUNT(*) - 1)
AS COVARIANCE FROM ELECTRONICS_DATA;
```

The resulting Covariance of **4.03** mathematically confirms the positive directional relationship between ratings and reviews observed in the slope calculation.